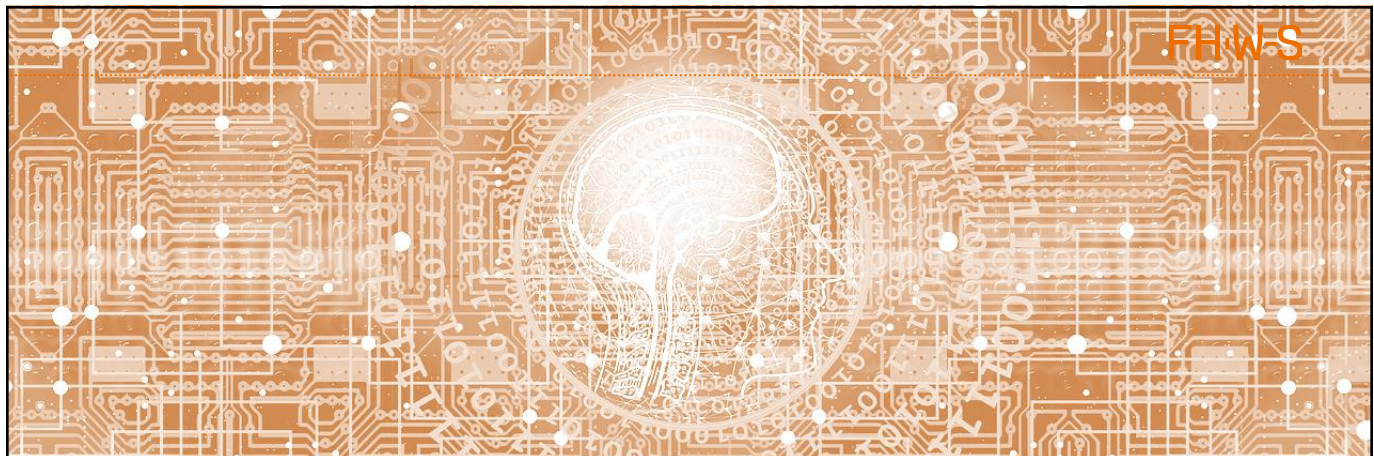




Software Engineering and Cyber Security

Prof. Dr. Marian Daun

FHWS University of Applied Sciences
Würzburg-Schweinfurt

The slide has a background image of a desk with a notebook, a pen, and a smartphone. A semi-transparent white box with an orange border contains the agenda items. The FHWS logo is in the bottom left, and the number '2' is in the bottom right.

Agenda

- Software Engineering Fundamentals
- Requirements Engineering
- Architecture Design
- Modelling
- Development and Implementation
- Software Quality Assurance

FHWS University of Applied Sciences
Würzburg-Schweinfurt

2

Modelling

Part D: Behavior

Fundamentals

Key Concept: State

Definition

- Defines a period of time in which the system behaves in a specific way.
- Waits for the occurrence of a defined event that triggers a state transition.

Notation

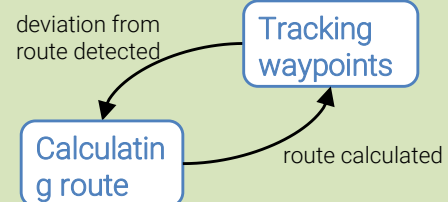
DFA/NFA



Statechart

[Name]

Example



Key Concept: Event

Definition

- Discrete incident in the world (external or temporal event).
- Occurrence of an event is instantaneous (duration = 0).
- An external event occurs in the system environment. The system is expected to produce a response.
- A temporal event is triggered when reaching a specific moment in time which triggers the system to perform some action.

Notation

DFA/NFA

event

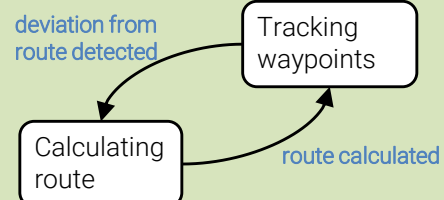


Statechart

event



Example

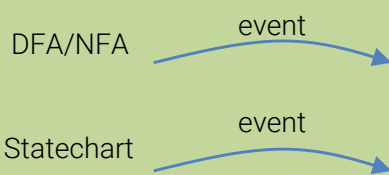


Key Concept: State Transition

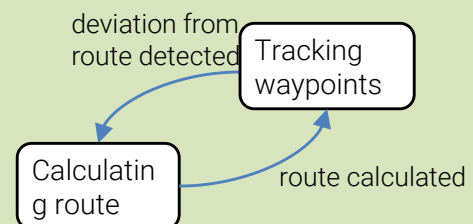
Definition

- Describes a change in the state of the system, i.e. the system changes from its current state to another state.
- An event triggers state transitions.

Notation



Example



Group Discussion

Modelling as State or Event?

Whether a particular aspect is modelled as a state or as an event, depends on the modelling purpose and the desired level of abstraction.

Finite Automata (DFA/NFA)

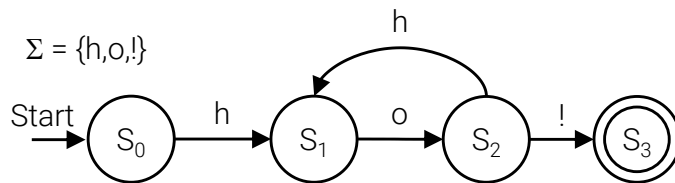
Deterministic Finite Automata (DFA)

A deterministic finite automaton is a five-tuple $(Q, \Sigma, \delta, q_0, F)$, where

- Q is a finite set of states.
- Σ is a finite set of input symbols (the input alphabet).
- δ is a transition function that takes as arguments a state s (from the set Q) and an input symbol a (from the set Σ) and returns a state s' (from the set Q).
- q_0 is the start or initial state.
The state q_0 is one of the states in Q .
- F is the set of final or accepting states.
The set F is a subset of Q .

based on [Hopcroft et al. 2007]

Partial DFA



For at least one state no transition is defined for at least one symbol of the input alphabet Σ .

Partial DFA: Tabular Documentation

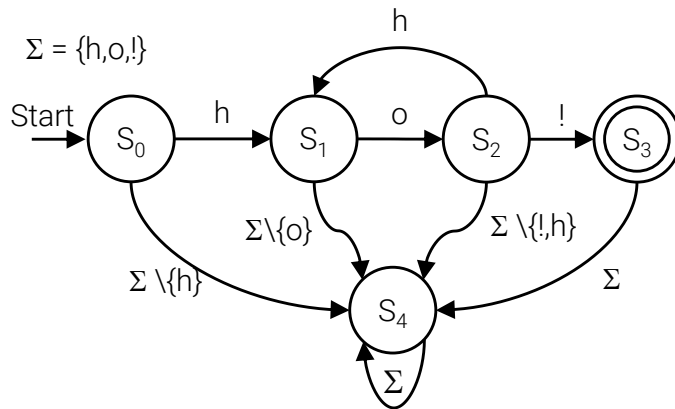
$(Q, \Sigma, \delta, q_0, F)$

	h	o	!	
$q_0 \rightarrow s_0$	s_1	?	?	$\leftarrow \Sigma$
s_1	?	s_2	?	
s_2	s_1	?	s_3	
$F \rightarrow *s_3$	?	?	?	

$\delta(s, s')$

Q

Complete DFA



For each state exactly one transition is defined for each symbol of the input alphabet Σ .

Complete DFA: Tabular Documentation

$(Q, \Sigma, \delta, q_0, F)$

	h	o	!	
$q_0 \rightarrow s_0$	s ₁	s ₄	s ₄	← Σ
s ₁	s ₄	s ₂	s ₄	
s ₂	s ₁	s ₄	s ₃	
$F \rightarrow *s_3$	s ₄	s ₄	s ₄	

Q

$\delta(s, s')$

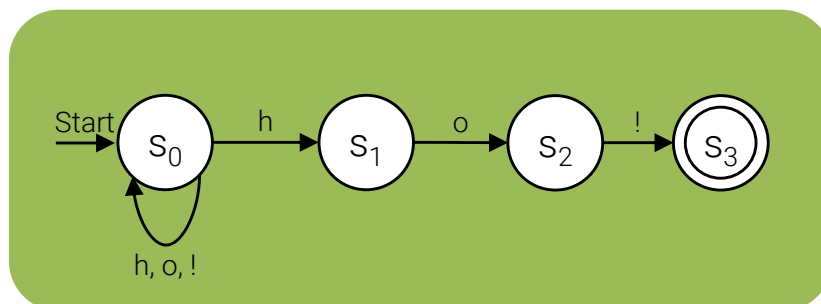
Non-deterministic Finite Automata (NFA)

A non-deterministic finite automaton is a five-tuple $(Q, \Sigma, \delta, q_0, F)$, where

- Q is a finite set of states.
- Σ is a finite set of input symbols (the input alphabet).
- δ is a transition function that takes as arguments a state s (from the set Q) and an input symbol a (from the set Σ) and returns a subset S of the states defined in Q .
- q_0 is the start or initial state.
The state q_0 is one of the states in Q .
- F is the set of final or accepting states. The set F is a subset of Q .

based on [Hopcroft et al. 2007]

Non-deterministic Finite Automata (NFA)



$$L(A) = (h+o+!)*ho!$$

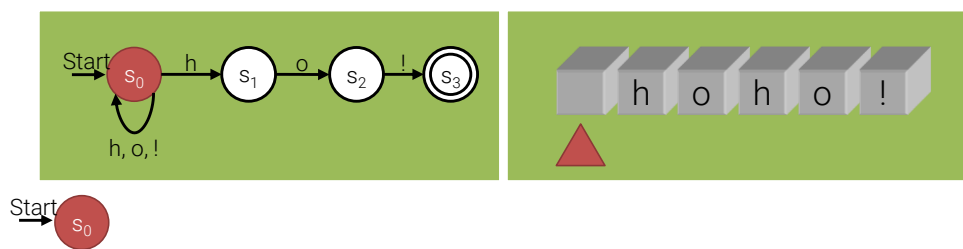
NFA: Tabular Documentation

		h	o	!	Σ
$q_0 \rightarrow$	s_0	$\{s_0, s_1\}$	$\{s_0\}$	$\{s_0\}$	
s_1	$\emptyset$	s_2	$\emptyset$		
s_2	$\emptyset$	$\emptyset$	s_3		
$F \rightarrow$	$*s_3$	$\emptyset$	$\emptyset$	$\emptyset$	

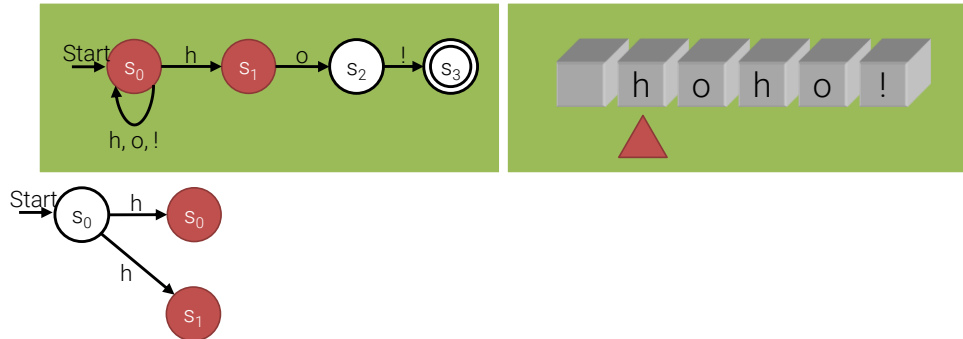
$(Q, \Sigma, \delta, q_0, F)$

$\delta(s, S)$

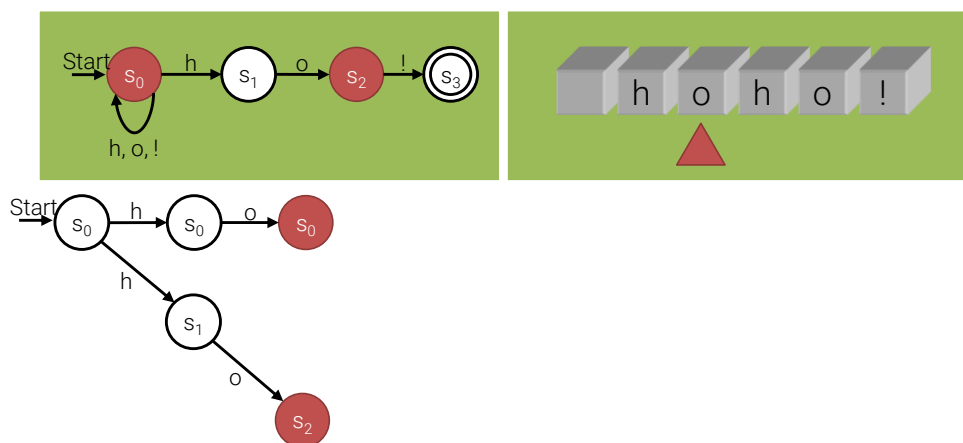
Non-deterministic Finite Automata



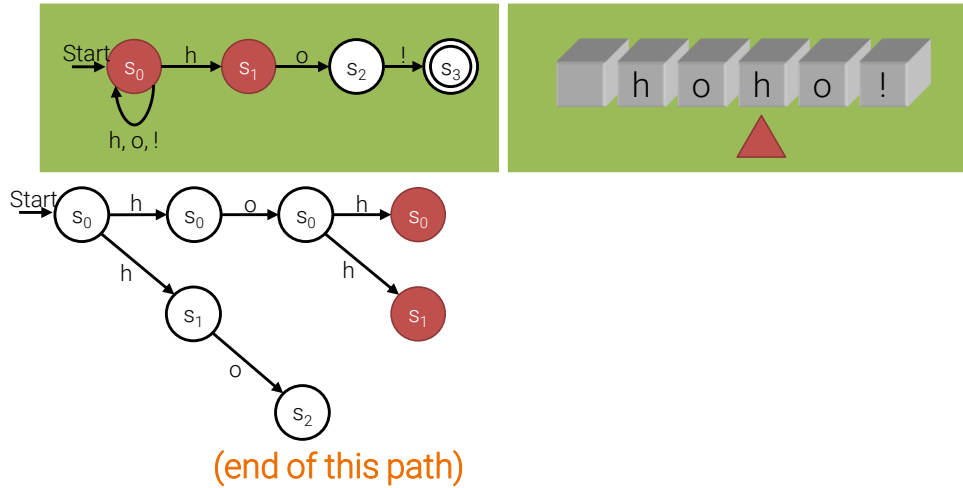
Non-deterministic Finite Automata



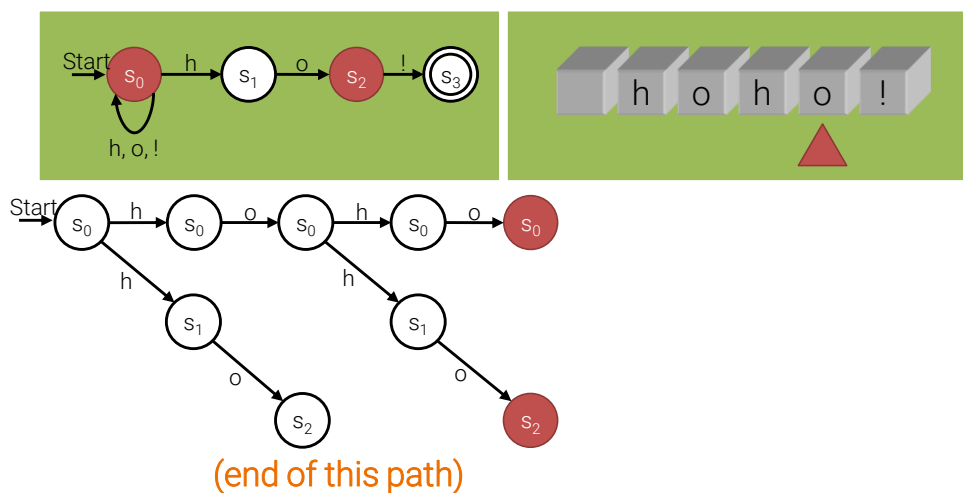
Non-deterministic Finite Automata



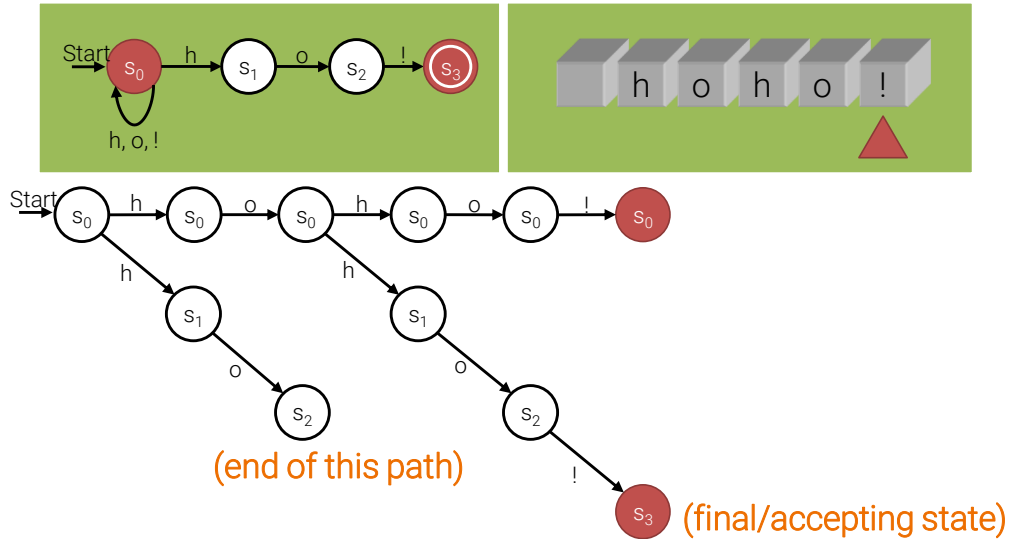
Non-deterministic Finite Automata



Non-deterministic Finite Automata



Non-deterministic Finite Automata



Group Discussion

What are Advantages of DFA compared to NFA? What are Disadvantages?

Statecharts

Key Enhancements

Statecharts provide means for:

- Hierarchically structuring the state space.
- Defining concurrent behaviour.
- Defining actions for states and transitions.
- Defining conditions for transitions.

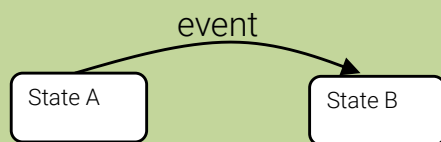
States and Transitions

Definition

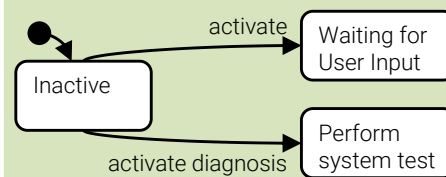
- A **state** represents a set of value combinations for the underlying element. It has a name, and may have an internal behaviour that is executed based on defined events
- A **transition** specifies a state change. It is a directed relationship between two states, and defines a trigger and a condition that both lead to the state transition, as well as a behaviour that is executed during the transition

[Weilkiens 2006]

Notation



Example



Conditional Transitions

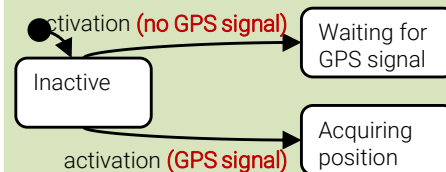
Definition

- Extending transitions with **guarding conditions**
- Transition is only performed when the corresponding event occurs **and** the condition is satisfied
- Condition is defined in brackets after the event that triggers the transition

Notation



Example

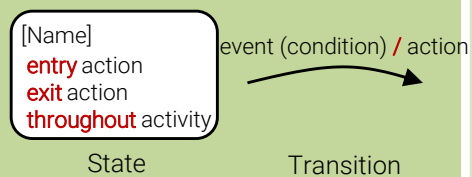


Activities and Actions

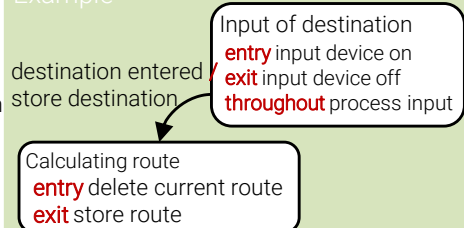
Definition

- Definitions of actions the system shall execute
 - When performing a state transition – "... / **action**"
 - On entering a specific state – "**entry**"
 - When leaving a state – "**exit**"
 - While being in the state – "**throughout**"

Notation



Example

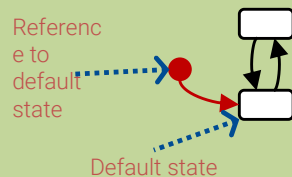


Default State

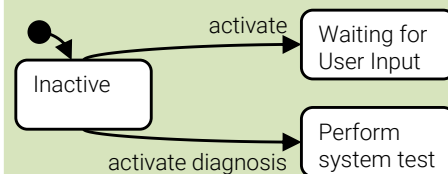
Definition

- Each statechart must have a **default state** (sometimes called "initial state") in which the system execution starts
- Note that statecharts provide a **reference** to the default state. This reference is no state of its own

Notation



Example

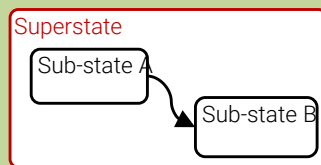


Hierarchical Refinement

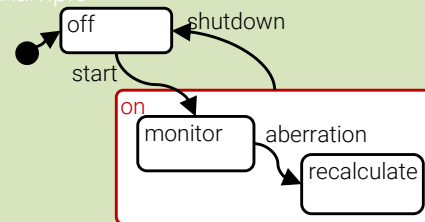
Definition

- Refinement of a state into a set of **sub-states**
- (or vice versa) “Aggregation” of several states into a **super-state**
- Super-state **encapsulates** part of the system behaviour
- “expand / collapse” details stepwise

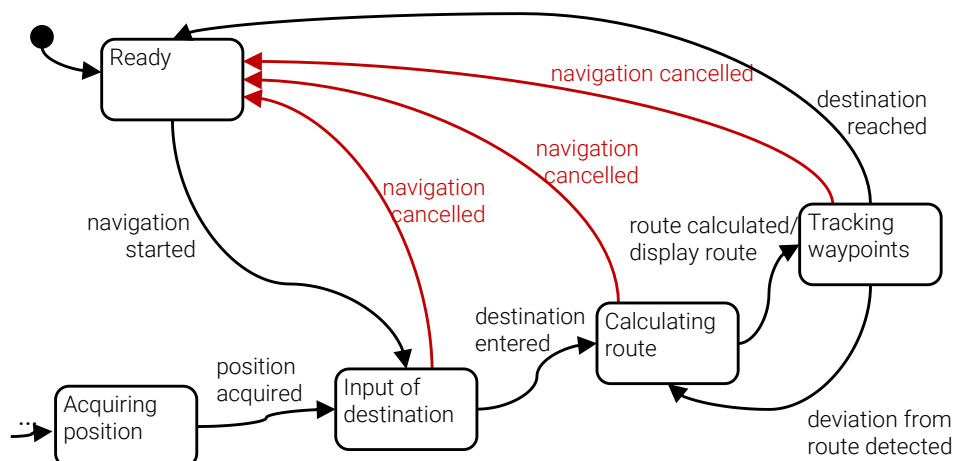
Notation



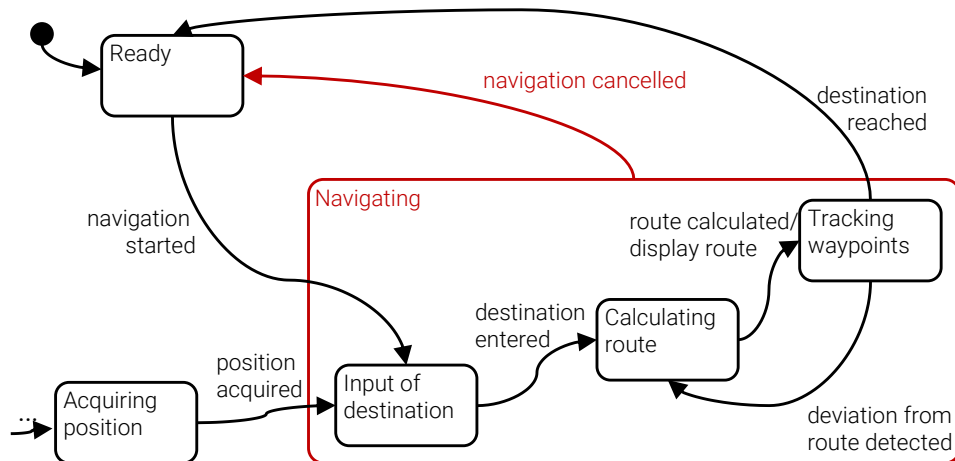
Example



Without Hierarchical Refinement



With Hierarchical Refinement

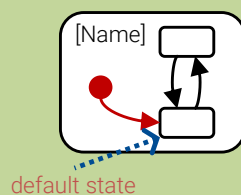


Hierarchical Refinement – Default State

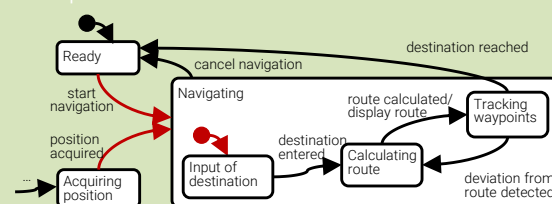
Definition

- For each super-state, one sub-state can be defined as the **default state**
- Default-state is entered whenever a transition is performed which activates the super-state and, for this transition, the sub-state to be activated is not directly defined

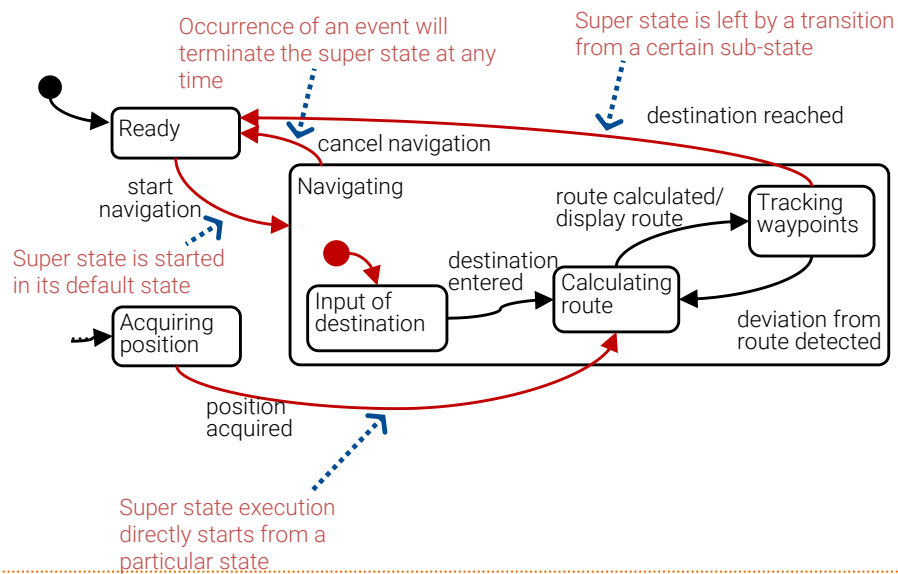
Notation



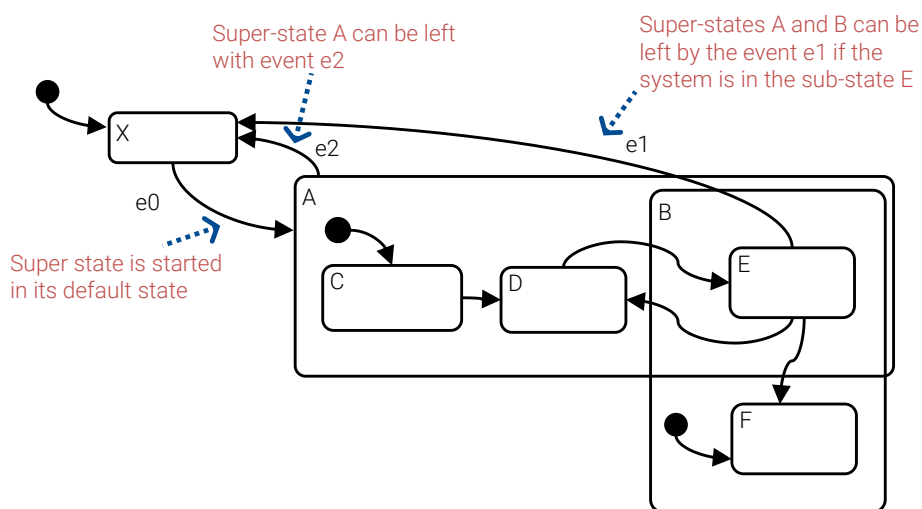
Example



Entry and Exits in Hierarchies



Overlapping State Hierarchies





Group Discussion

Personal Opinion: Use Overlaps sparsely
– or even better: not at all

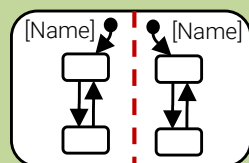
Why?

Concurrency

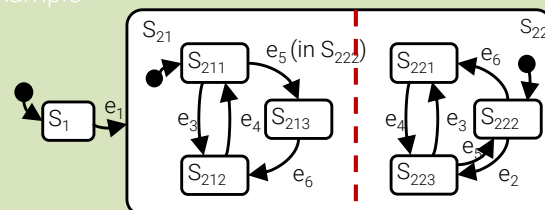
Definition

- A super-state can be decomposed into several **concurrent states**
- If the system enters a super-state, for which concurrent states are defined, the system enters all concurrent states at the same time
- A concurrent state can again be redefined, i.e. for each concurrent state sub-states and transitions can be defined

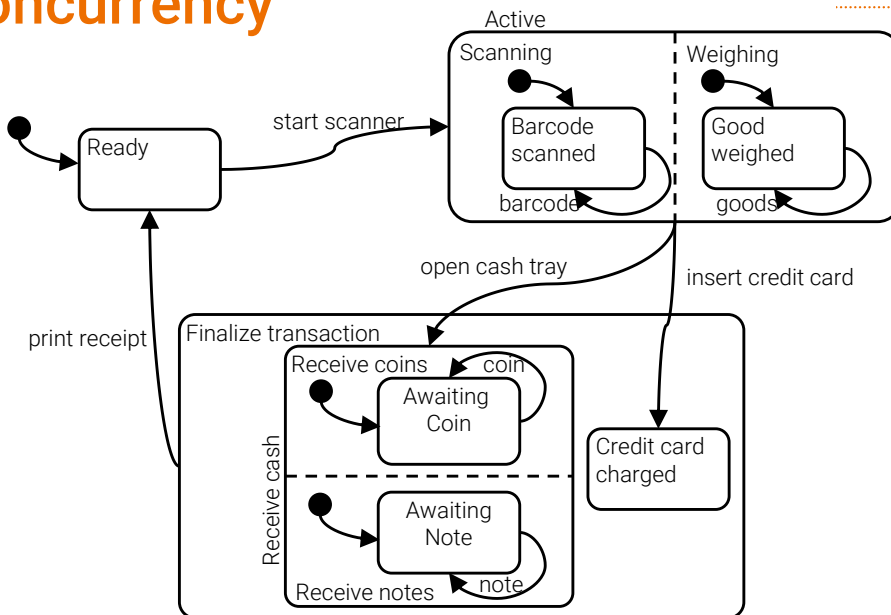
Notation



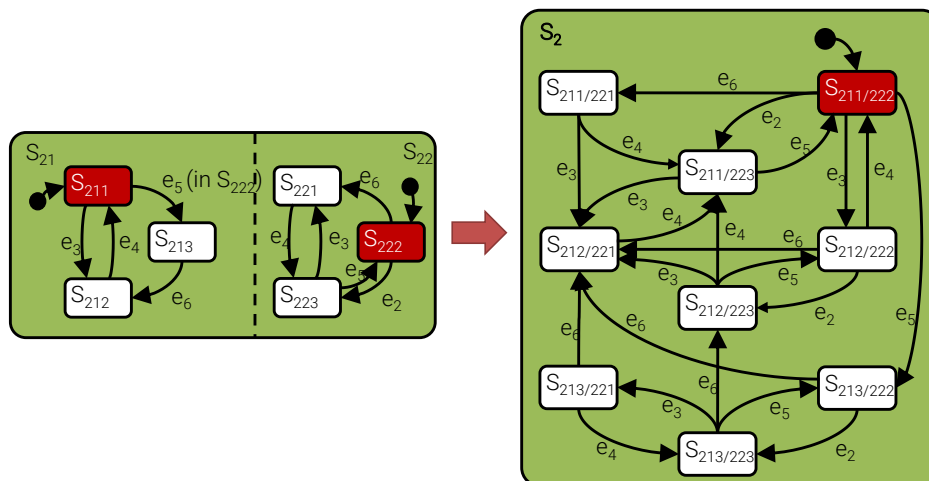
Example



Concurrency



Flattening Concurrency in Statecharts

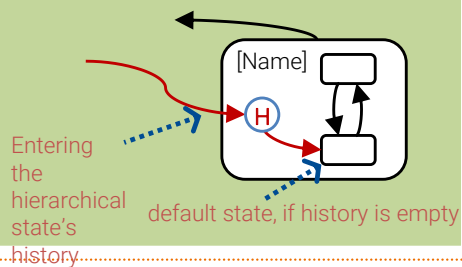


History State

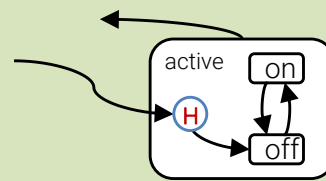
Definition

- **History States** memorize the state of a super-state when the super-state is left.
- If the super-state is entered again the last visited state will be resolved and not the default state

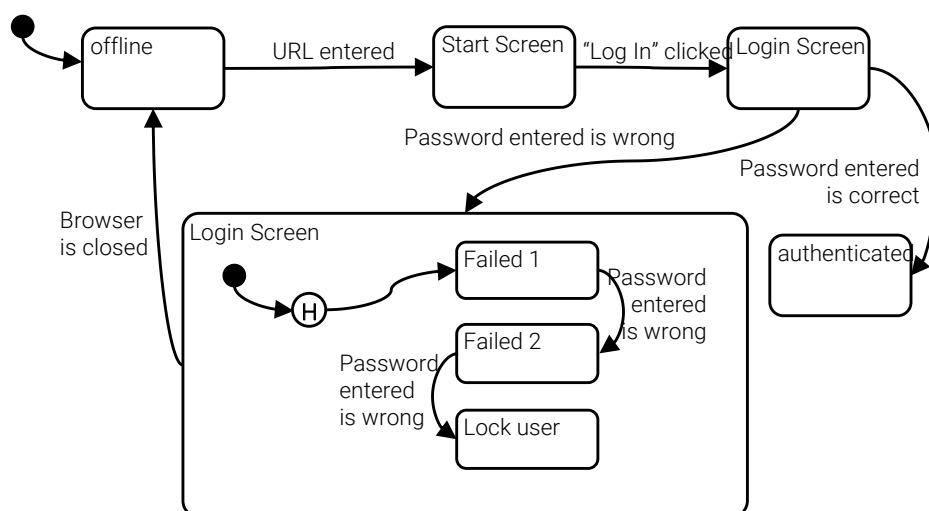
Notation



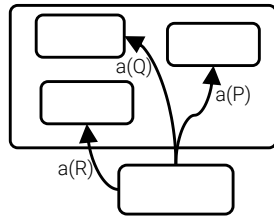
Example



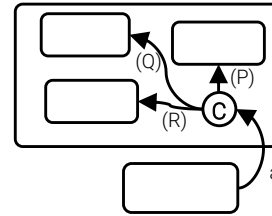
History



Conditions



Conditional Transitions



Condition State



Exercise

Create a Statechart for a
fully automatic coffee machine

UML State Machine Diagrams

Purpose

UML State Machine Diagrams are based on statecharts.

Statecharts use a specific notation, state machine diagrams represent statecharts in the common notation formats of the UML.

Note: In minor parts, enhancements of statecharts and statemachine diagrams differ not only in the notation. For example:

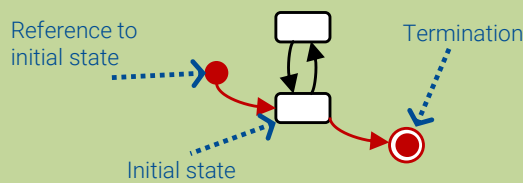
- entry and exit of hierarchical states
- final states
- submachine diagrams

Initial and Final State

Definition

- Each state machine diagram must have an **initial state** in which the system execution starts.
- In addition, and different from statecharts, state machine diagrams can have **final states** representing the **termination of system execution**.

Notation



Example

FH·W-S

University of Applied Sciences
Würzburg-Schweinfurt

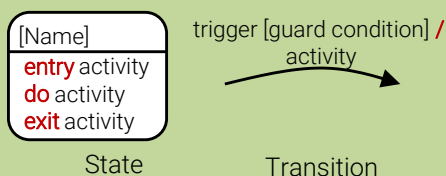
47

Modelling Construct: Activities

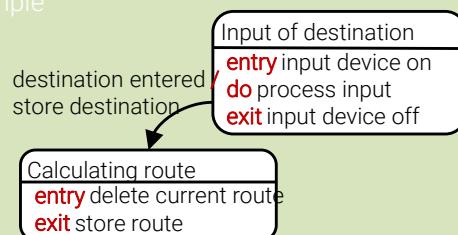
Definition

- Definitions of activities the system shall execute
 - When performing a state transition – “... / **activity**”
 - On entering a specific state – “**entry**”
 - While being in the state – “**do**”
 - When leaving a state – “**exit**”

Notation



Example



FH·W-S

University of Applied Sciences
Würzburg-Schweinfurt

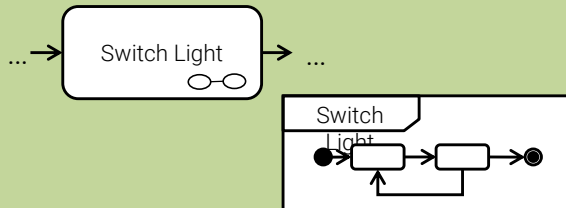
48

Key Enhancement: Submachine State

Definition

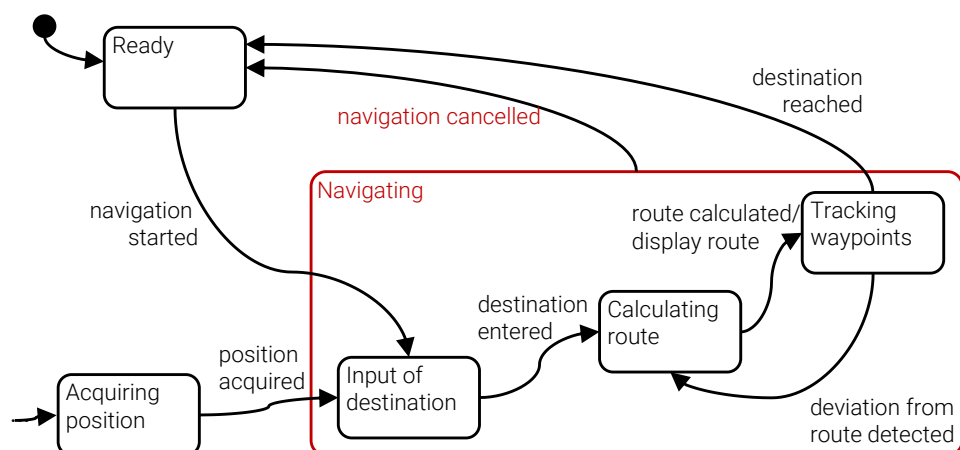
- A submachine state refers another state machine.
- Definition of submachine states supports the reuse of a state machine within other state machines.

Notation

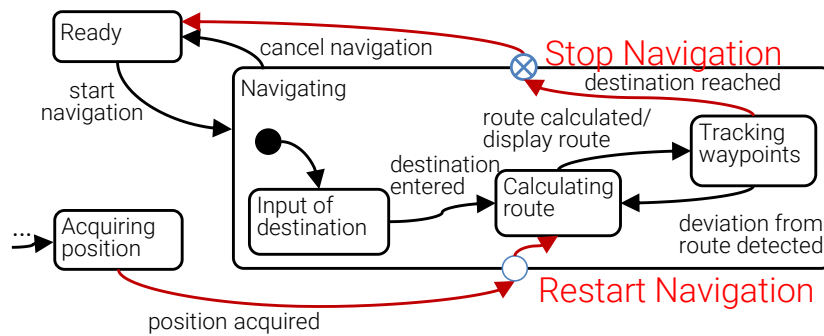


Example

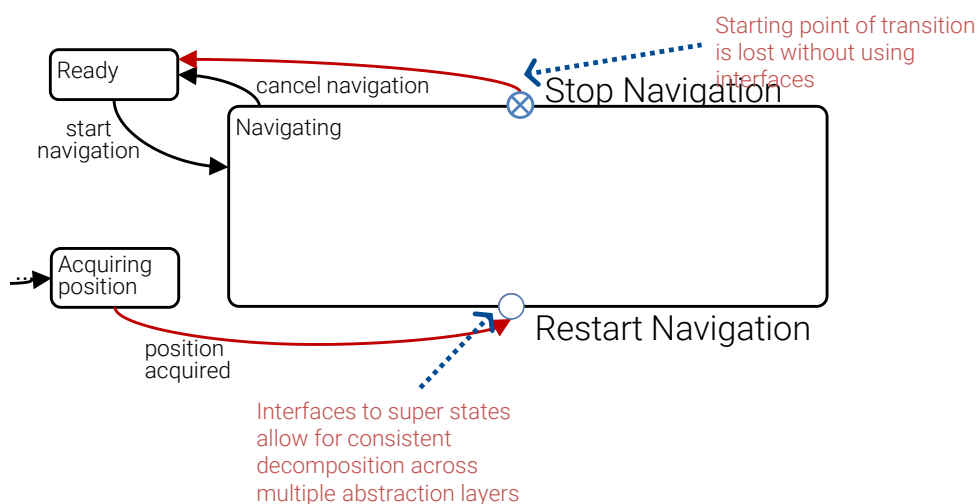
Hierarchy vs. Submachine States



Entry and Exit Points



Entry and Exit Points

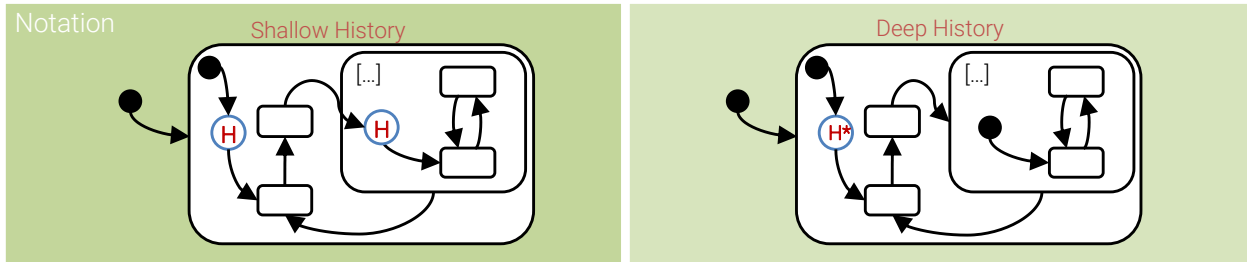


Shallow History and Deep History

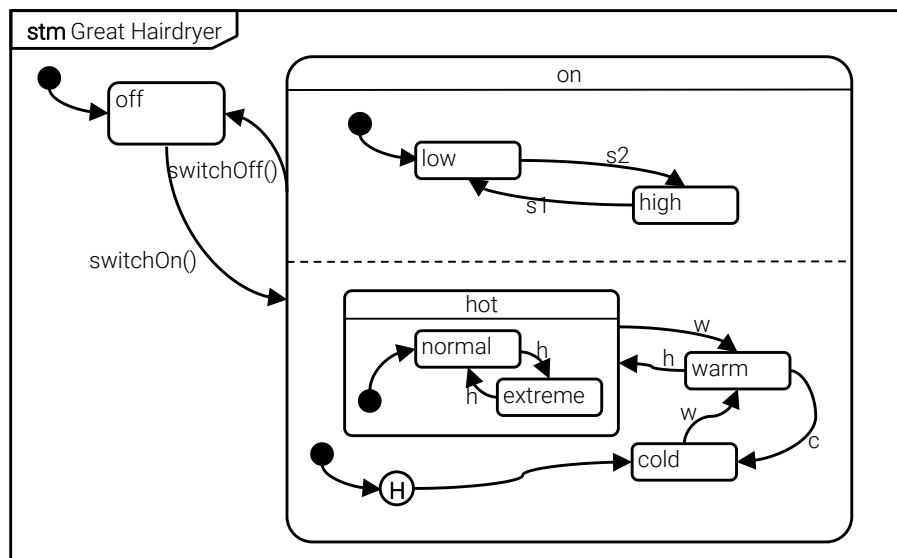
Definition

- UML distinguishes the **history operator** into two types: shallow history and deep history.
- Shallow history** stores the history of the selected state in one super-state. (same as Statecharts)
- Deep history** also memorizes the history within substates of the super state.

Notation

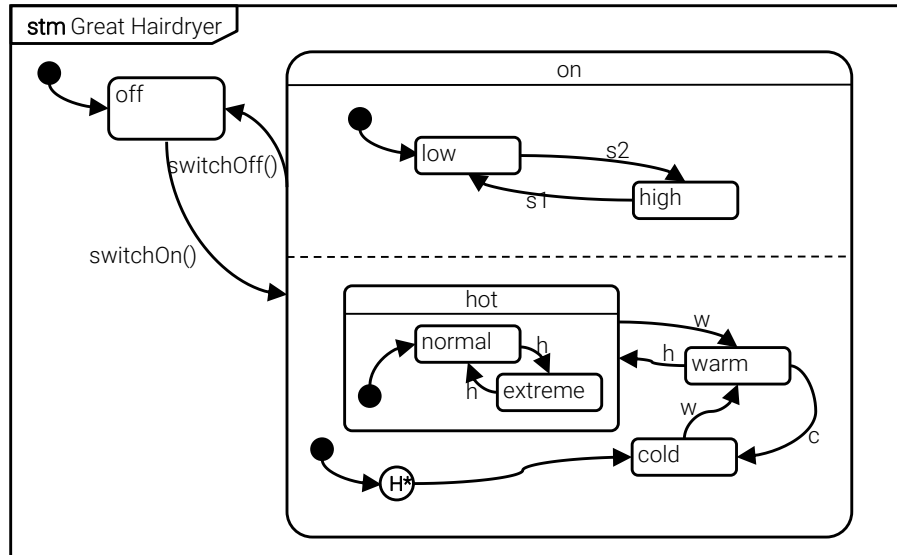


Shallow History



[Weilkiens 2006, p. 200]

Deep History



[Weilkiens 2006, p. 200]



Group Discussion

Precision vs Complexity?

Literature

- [Davis 1993] A.M. Davis: Software Requirements – Objects, Functions, and States. 2nd ed., Prentice Hall, 1993.
- [Harel 1987] D. Harel: Statecharts – A Visual Formalism for Complex Systems. In: Science of Computer Programming, Vol. 8, No. 3, 1987, pp. 231-274.
- [Hopcroft et al. 2007] Hopcroft, Motwani, Ullman: Introduction to Automata Theory, Languages, and Computation. 3rd ed., Pearson, 2007.
- [Mealy 1955] G. H. Mealy: A Method for Synthesizing Sequential Circuits. In: Bell System Technical Journal, Vol. 34, No. 5, 1955, pp. 1045-1079.
- [Moore 1956] E. F. Moore: Gedanken-Experiments on Sequential Machines. In: C. Shannon, J. McCarthy (Eds.): Automata Studies, Princeton University Press, Princeton, 1956, pp. 129-153.
- [Weilkiens 2006] T. Weilkiens: Systems Engineering with SysML/UML: Modeling, Analysis, Design. Morgan Kaufmann, 2008.

Questions for Self-Assessment

- What are the modelling constructs of finite automata?
- Which differences exist between behavioural modelling languages?
- What are the modelling constructs of Statecharts?
- What are the modelling constructs of UML State Machine Diagrams?
- How do you decide which modelling language is suitable for the given purpose?
- How do you decide which modelling constructs you use?

Thanks

UNIVERSITÄT
DUISBURG
ESSEN

Offen im Denken

PALUNO
The Ruhr Institute for Software Technology
<https://www.paluno.uni-due.de>

This Lecture Material is based on Lecture Material provided by
Prof. Dr. Klaus Pohl from **University of Duisburg-Essen**

The original Lecture Material was created by:

Prof. Dr. Klaus Pohl

Torsten Bandyszak

Jennifer Brings

Johanna Buchner

Prof. Dr. Marian Daun

Felix Föcker

Andrea Salmon

Philipp Schmidt

Prof. Dr. Bastian Tenbergen

Prof. Dr. Thorsten Weyer